

CyberSentinel

Backend Redesign — Task Execution for ASM / VS / CA

A queue-driven design where **Python (FastAPI)** owns everything the user touches, the **Go worker** does execution only, **Redis** is the nervous system (concurrency slots + live state + events), and **Postgres** is durable memory (run state + findings). The three services — ASM, VS, CA — share one identical flow; only their stage list and tools differ.

Reference kept from the current system: the ASM pipeline (stage design) and the tool wrappers. Everything else is redesigned.

1 · Component responsibilities

Python / FastAPI	User-facing surface: trigger scans, scheduler/cron, task command & query API, live progress stream (SSE/WebSocket), reporting consumer, notifications.
Go worker	Execution only. Consume job, admit via Redis slot, build pipeline, run tools concurrently, write findings + progress, push to reporting queue. Stateless, horizontally scalable.
RabbitMQ	Transport. Per-service, per-priority job queues (asm/vs/ca . high/medium/low) with a dead-letter queue each, plus a reporting queue.
Redis	Concurrency slots (global limit), live task/pipeline state, progress pub/sub, cancel flag, worker lease/heartbeat.
Postgres	Durable run/task state and all findings, written incrementally during the run.

Key decision: no separate Go 'Gin' control-plane. Admission is an atomic Redis operation; task state and the command API live in FastAPI + Postgres. One source of truth, no synchronous hop in the hot path.

2 · Task lifecycle (one task, end to end)

1	Trigger	User clicks Run (or a scheduled/auto run fires). FastAPI writes the task row as PENDING and publishes one message to the matching queue. The cron does the same on time-match.
2	Enqueue	Message lands in asm.high / vs.low / etc., carrying task_id, org_id, target/asset, mode and config.
3	Consume	The Go worker for that service picks the message from its queue.
4	Admit	Worker acquires a slot in Redis (atomic, global max per service). Got a slot → ADMITTED; none free → backpressure (message waits).
5	Build	Worker loads asset data, selects the stage list (the ASM pipeline reference), and writes the task JSON (stages + result slots) to Redis. State → RUNNING.

6	Run stages	In a goroutine, for each stage: resolve tool from registry → run → save findings to Postgres immediately → update progress in Redis → publish a progress event → check cancel flag → refresh lease.
7	Live UI	FastAPI subscribes to Redis pub/sub and streams events to the browser (SSE/WebSocket, org-scoped). User sees looking → finding → done per stage.
8	Finish	Last stage done → task COMPLETED, Redis slot released, message pushed to the reporting queue, goroutine ends.
9	Report	Python reporting consumer builds the report and fires a notification. Findings are already visible (from step 6).

Safety paths: Cancel — FastAPI sets task:{id}:cancel; the worker stops cleanly between stages → CANCELLED. Crash — the Redis lease expires; a reaper marks the task FAILED or requeues it and frees the slot.

3 · Contracts (lock these before code)

Queue topology (RabbitMQ)

```
Job queues:  asm.high  asm.medium  asm.low
             vs.high  vs.medium  vs.low
             ca.high   ca.medium  ca.low
Each queue:  x-dead-letter-exchange -> <name>.dlq
Reporting:   reporting  (+ reporting.dlq)
```

Job message (Python publishes → Go consumes)

```
{
  "type":      "asm|vs|ca",
  "priority":  "high|medium|low",
  "task_id":   "uuid",
  "org_id":    "uuid",
  "asset_id":  "uuid|null",
  "targets":   [...],
  "mode":      "LIGHT|NORMAL|DEEP",    // asm intensity / vs|ca scan mode
  "config":    { }
}
```

Task lifecycle states (Postgres + Redis agree)

```
PENDING -> ADMITTED -> RUNNING -> COMPLETED | FAILED | CANCELLED
```

Redis keys

slots:{service}	integer, atomic acquire/release vs configured max (global concurrency)
task:{task_id}	JSON: {status, current_stage, stages:[{name,tool,status,result}], started_at}
task:{task_id}:cancel	set to request cancellation
task:{task_id}:lease	TTL heartbeat; reaper requeues if it expires while RUNNING
task_events:{org_id}	pub/sub channel; worker publishes, FastAPI SSE subscribes

Postgres

Reuse asm_discoveries + asm_discovery_runs for durable run/task state (add generic columns if VS/CA need them). ASM finding tables already exist (subdomains, ips, ports, services, ssl, endpoints...); add equivalent finding tables for VS and CA when those services are built.

4 · Worker folder structure (lean)

One shared engine; each service is essentially its stage list + how it saves. Tool wrappers are one thin file each (they shell out to real binaries).

```

worker/
|- main.go          boot, register asm/vs/ca, start consuming
|- config.go        env + pg + redis + rabbit connections
|- queue.go         consume/ack/nack(DLX) + publish + worker pool + drain
|- pipeline.go      THE engine: run stages -> redis state -> emit events -> cancel -> save hook
|- tools/
|   |- registry.go  name -> run func (self-registering)
|   |- exec.go      shared LookPath + Run (one place; no getToolPath copies)
|   \- subfinder.go httpx.go nmap.go ... one thin file per tool
\-- services/
    |- asm.go       stage lists (per mode / asset-type) + SaveFindings
    |- vs.go        same shape, different stages + tools
    \- ca.go        same shape

```

A service answers only two questions: **what stages, in what order?** and **how do I save findings?** The engine (consume, admit, run loop, Redis state, events, cancel, reporting hand-off) is written once and shared by all three.

Service files are structurally identical

asm.go, vs.go and ca.go implement the same Service interface — the same members, in the same order, with the same signatures. Only the bodies differ. A reviewer diffing asm.go against vs.go must see the exact same skeleton, different logic inside.

```

type Service interface {
    Name() string           // "asm" | "vs" | "ca"
    Queues() []string       // asm.high, asm.medium, asm.low
    Stages(job Job) []Stage // pick stage list by mode / asset-type
    SaveFindings(ctx, task, stage, out) error
}

// each file, same order:
// 1 type 2 init()->RegisterService 3 Name 4 Queues
// 5 Stages (the only place the pipeline differs) 6 SaveFindings 7 private helpers

```

Invariant: no service adds a public method the others lack, and no service-specific logic leaks into pipeline.go — the engine only ever calls the four interface methods.

5 · Phased build plan

Every phase compiles (go build ./... & go vet ./...) and is independently testable. No big-bang rewrite.

0	Contracts	Lock queue names, message schema, task states, and the Redis key schema (Section 3). Everything else depends on these.
1	Skeleton	main.go + config.go + queue.go + pipeline.go engine + tools/{registry,exec}; services/asm.go with a no-op pipeline. Builds green.
2	Concurrency	Redis slot acquire/release (atomic), task state in Redis, durable run-state writes to Postgres, lease + cancel plumbing.
3	ASM pipeline	Port the existing ASM stage config (reference). Engine runs stages via the tool registry; findings saved to Postgres incrementally.
4	Live events	Worker publishes stage events to Redis pub/sub; FastAPI SSE/WebSocket streams to the UI (org-scoped).

5	Reporting	On completion push to the reporting queue; Python reporting consumer builds report + notification.
6	Schedule / auto	Python scheduler publishes to queues on cron match; auto = fixed-cadence rescan.
7	Resilience	Stale-task reaper (lease expiry), end-to-end cancellation, DLQ handling + bounded retries.
8	VS then CA	Copy the ASM service template; swap the stage config + tools/adapters. Same engine, same lifecycle.

Definition of done

One real ASM job flows end to end: **enqueue** → **admit** → **pipeline** → **live events** → **findings in Postgres** → **reporting** → **notification** — with working cancel and crash recovery. Then VS and CA are added by cloning the ASM service with different stages and tools.